

Data Engineering and Machine Learning Pipeline

Phase II Project Report

ENGE707 Data Engineering and AI

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Content

1. Introduction – Project Phase-I Summary.....	4
1.1 Problem Restatement.....	4
1.2 Dataset summary.....	4
1.3 Data-quality issues identified in Phase I.....	5
1.4 Justification for an LCS-based solution.....	6
2. Original LCS System on Raw Dataset.....	7
2.1 LCS variant selection (preferably eLCS) – justification.....	7
2.2 Confirmation that original LCS code is used without algorithmic modification.....	7
2.3 Execution on raw / minimally processed dataset.....	8
2.4 LCS parameter settings used.....	10
2.5 Baseline performance results.....	11
2.6 Minimal processing required for LCS compatibility.....	12
3. Data Preprocessing and Feature Engineering.....	14
3.1 Data leakage prevention measures.....	14
3.2 Data cleaning.....	15
3.3 Outlier investigation and handling.....	17
3.4 Feature engineering.....	18
3.5 Encoding categorical variables.....	19
3.6 Discretising & Encoding numerical variables.....	20
3.7 Class-imbalance handling.....	21
3.8 Expected impact of preprocessing on LCS performance.....	21
4. Improved LCS-Based System.....	22
4.1 Summary of changes vs. the original LCS system.....	22
4.2 Justification.....	24
4.3 Pseudocode of the improved system.....	25
4.4 Comparison against the original LCS baseline.....	27
5. Experiments.....	29
5.1 Validation strategy, Justification of using 80/20 split over k-fold CV.....	29
5.2 Evaluation of metrics DO NOT TOUCH >CCCCC.....	30
5.3 Statistical significance testing DO NOT TOUCH >CCCCC.....	32
6. Model Comparison.....	33
6.1 Original LCS – raw/minimally processed data.....	33
6.2 Original LCS – preprocessed data.....	34
6.3 Improved LCS – preprocessed/feature-engineered data.....	34
6.4 Chosen three conventional models.....	34
6.5 Model Setup.....	35
7. Interpretation of Results and Explainable AI, leo cao :3.....	39

7.1 Interpretation of main findings from the model comparison.....	39
7.2 Explanation of key LCS rules / feature conditions / classifier patterns.....	39
7.3 At least three sample LCS rules discussed for decision-making insight.....	39
7.4 How rule-based outputs support interpretability.....	39
7.5 Trustworthiness and practical meaningfulness of the explanations.....	39
8. Discussion, Limitations, and Future Work.....	39
8.1 Reasons for performance improvement (or lack thereof).....	40
8.2 Practical significance of results beyond raw numbers.....	40
8.3 Limitations – data quality, dataset size, class imbalance, noise, missingness, feature relevance.....	40
8.4 Future work – improved LCS variants, better preprocessing, external validation, domain expert review.....	40
References.....	40

1. Introduction – Project Phase-I Summary

1.1 Problem Restatement

Heart disease is a major concern that currently affects 195,000 people in New Zealand alone. Approximately every 90 minutes, a New Zealander dies from heart disease, and it is the number one killer for women (*Latest Heart Disease Statistics*, n.d.). In this project, the dataset “281K Patient Dataset for Multi-Disease Prediction” from Kaggle is used. The goal is to find whether the lifestyle, demographics, and clinical measurements in the dataset have any relevance to predicting whether an individual has heart disease. A model with this prediction ability isn’t a full replacement for having an in-person cardiovascular risk assessment. Still, it could be a cost-effective way to check for an

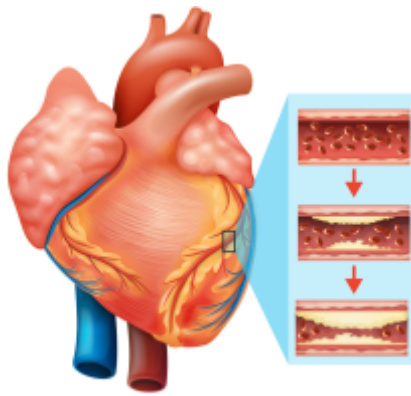


Image: © bluringmedia/Getty Images

individual’s chance of having heart disease. The target column “heart_disease_target” is a binary classification created after cleaning and is based on the “sublabel” column, which contains the combination of diseases an instance has. If the instance from the sublabel contains “HT”, the value of the new column would be True; otherwise, False. This new column is created to be easily used to compare and train the machine learning model’s prediction capabilities.

1.2 Dataset summary

Unified Chronic Disease Dataset (281K Patients, 38 Features)

Overview

The **Unified Chronic Disease Dataset** is an integrated healthcare dataset developed by combining three publicly available datasets covering **diabetes, heart disease, and hypertension**. The integration was performed using a **Medallion Architecture (Bronze → Silver → Gold)** in Microsoft SQL Server to create a clean, standardized, and analytics-ready dataset for machine learning and healthcare research.

The final dataset contains **280,985 anonymized patient records** with **38 clinical and lifestyle features** and **no missing values**. Each record represents an individual patient’s demographic profile, health measurements, lifestyle factors, and disease indicators.

To support a wide range of predictive modeling tasks, the dataset provides both a **binary target (Normal or Abnormal)** and an **8-class disease-combination label**, enabling researchers to study individual diseases as well as co-morbidity patterns.

Figure 1.2.1: Dataset overview from Kaggle

The dataset is a combination of three datasets (diabetes, heart disease, and hypertension). There are 280,985 raw records, and 278,701 remain after removing duplicates; the 278,701 will be used instead of the raw amount. The original dataset has 39 columns, but the introduction of “heart_disease_target” gives 40 columns. The columns utilised during the analysis and processing stage are lifestyle, demographics, clinical, and disease indicators. No missing values were found during the dataset inspection.

The overall heart disease rate is 2.1% across all these sources, but individually the dataset shows varying heart disease rates: 20.06% in the heart disease dataset source, compared to 4.17% for the diabetes source, and 0% for the hypertension source.

1.3 Data-quality issues identified in Phase I

From the data inspection and data cleaning stage, multiple issues in the dataset were identified:

- 2,284 duplicate instances were removed.
- heart_disease has fractional values (from 0 to 1), which doesn't clearly explain whether an individual has heart disease, so a new column (heart_disease_target) was created to serve as the target (binary) instead of the heart_disease column.
- The columns sublabel, disease_flags, label, and heart_disease all contain data leakage information. This provides clues and partial answers to whether the individual has heart disease; thus, they are excluded from the prediction process.
- Some columns such as age/age_level/age_normalized and bmi/bmi_level carry the same information; this is to be reviewed before modelling.
- Imbalance in the distribution of instances with and without heart disease, since 97.9% of records do not have heart disease and 2.1% do.
- As described in the dataset summary, individual sources have different heart disease rates.

1.4 Justification for an LCS-based solution

From the exploratory data analysis in phase 1, the heat map demonstrated that there was a weak linear correlation with the heart_disease_target; e.g age is 0.08, BMI is 0.04, and triglycerides are 0.09. This suggests that a single feature alone does not have a strong correlation with the target. When just analysing BMI level and age group, a clear difference in heart disease rate was observed between different age groups and BMI levels; seniors who are also overweight displayed the highest heart disease rate of 6.53% compared to 2.1% of the overall heart disease rate. Adding gender results in a difference in heart disease rates of 8.18% for males and 4.99% for females who are seniors and overweight. The overall takeaway is that single features are weak as linear predictors, and that combining specific features together showed a greater heart disease rate.

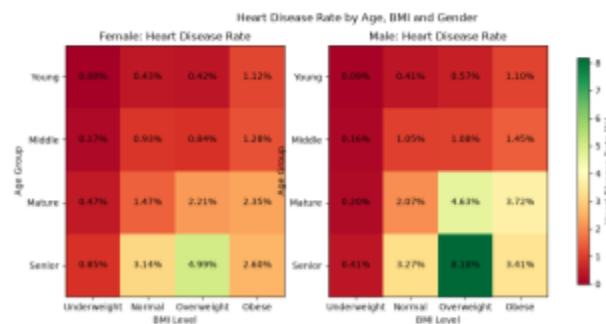


Figure 1.4 Heart disease rate across combinations of age, BMI and gender from phase 1

The main reason why LCS is suitable for this problem is that it allows us to find a specific set of conditions that predicts the heart_disease_target well. In the exploratory data analysis, all possible combinations of features with the target could not be examined due to time constraints as well as the number of possible combinations of features in the dataset; therefore, LCS would help with automation. Another reason is that LCS involves a population to create if-then rules (classifiers). This could allow interpretability for each classifier, as the produced model aims to help predict heart disease. Experts in the field could analyse these attributes to see whether each classifier makes medical sense. However, there is a challenge: the dataset has a clear imbalance of instances with heart disease compared to those without; thus, the LCS machine learning model may favour the majority class. Further evaluation would be needed beyond just the accuracy.

2. Original LCS System on Raw Dataset

2.1 LCS variant selection (preferably eLCS) – justification

We used eLCS, also known as Education Learning Classifier System, as our LCS system for this project. eLCS is a variant of Learning Classifier systems that uses supervised learning and is used for classification problems. This project's goal is to successfully predict whether a patient has heart disease, which is represented by "heart_disease_target" with a value of True or False. A True/False value can be represented by binary 1 for True and 0 for False, making it a binary classification problem.

In our dataset, "281K patient dataset", there are both continuous and categorical variables. eLCS can work with both of these types of variables, which made it a suitable LCS for this project.

We learned in phase I that there were no relationships between individual features and the target attribute. However, after combining features together and then comparing them, the relationships were clearer. eLCS is capable of learning rules that include a combination of attributes, which is exactly what this project requires for a machine learning model.

2.2 Confirmation that original LCS code is used without algorithmic modification

```
setup(  
    name = 'scikit-eLCS',  
    packages = ['skeLCS'],  
    version = '1.2.4',  
    license='License :: OSI Approved :: GNU General Public License v3 (GPLv3)',  
    description = 'Educational Learning Classifier System',
```

Figure 2.2.1: eLCS version confirmation

Our original dataset was given to the original eLCS implementation. The version used was 1.2.4, and there were no modifications made to the source code or behaviour of the system. The setup remained untouched and was found in the “setup.py” file in the system files. External code was added outside the eLCS implementation package, but this was just to make the dataset ready for model training. The following code was added:

1. Loading the dataset
2. Creating the target attribute
3. Splitting the dataset into training and testing sets (80/20 split)
4. Predicting the results
5. Calculating all of the metrics from model training

2.3 Execution on raw / minimally processed dataset

```
Raw shape: (280985, 39)
Raw records: 280985
Raw columns: 39
Missing values: 0
Duplicate records: 2284
```

Figure 2.3.1: Raw Dataset

The raw dataset from “patient_data.csv” was used for the baseline eLCS model. As shown in Figure 2.3.1, there were 280,985 records and 39 columns. The raw dataset contained 2284 duplicate records, which were intentionally kept. The duplicate rows will be handled during Task 3 because the original eLCS model was meant to represent the raw dataset with none to very minimal preprocessing.

```
heart_disease_target
0    275081
1     5904
Name: count, dtype: int64
```

heart_disease_target	0	1
sublabel		
DI	31918	0
DI_HT	0	1398
DI_HT_HY	0	845
DI_HY	66473	0
HT	0	2595
HT_HY	0	1066
HY	69806	0
N	106884	0

Figure 2.3.2: Target Construction

The target attribute was not in the original dataset, so it was added after loading. The “sublabel” attribute was used to create it by checking if the value contained “HT”, which represented heart disease. The values including “HT” were mapped to class 1, while the values not including it were mapped to class 0.

The result of the target creation was that 275,081 records were mapped to class 0 (no heart disease), and 5,904 records were mapped to class 1 (heart disease). With these results, the positive class rate was approximately 2.10%.

```
Predictor shape: (280985, 36)
Target shape: (280985,)
Positive-class rate: 0.021011797782799792
```

Figure 2.3.3: Predictor Set

After the target attribute was added, there were a total of 40 columns. However, 4 of these were excluded from the predictor matrix due to having a one-to-one correlation with the target which would result in data leakage.

Therefore, the baseline experiment used 36 columns. We performed validation checks which confirmed the target attribute was binary and the dataset contained no missing values.

```
Training shape: (224788, 36)
Testing shape: (56197, 36)
Training positive rate: 0.021010908055590156
Testing positive rate: 0.021015356691638343
```

Figure 2.3.4: Train/Test sets

The dataset was split into training and testing sets with an 80/20 split. Stratification was used to preserve the class imbalance in both sets so that they could both be as close to the original dataset split as possible. The results shown in Figure 2.3.4 tell us that the training

set contained 224,788 rows (80% of rows from the dataset) and the testing set contained 56197 rows (20% of the dataset). Both sets used 36 attributes because 4 columns from the original dataset had a one-to-one correlation with the target attribute. Both sets maintained the same positive rate of approximately 2.10%, which was the same rate in the raw dataset.

```
Starting seed 3..
Completed seed 3: balanced accuracy=0.5000, ROC-AUC=0.6925, PR-AUC=0.2241, positive predictions=0
Starting seed 6..
Completed seed 6: balanced accuracy=0.5000, ROC-AUC=0.7354, PR-AUC=0.1711, positive predictions=0
Starting seed 9..
Completed seed 9: balanced accuracy=0.5000, ROC-AUC=0.6086, PR-AUC=0.0782, positive predictions=0
Starting seed 12..
Completed seed 12: balanced accuracy=0.5000, ROC-AUC=0.6885, PR-AUC=0.1787, positive predictions=0
Starting seed 15..
Completed seed 15: balanced accuracy=0.5000, ROC-AUC=0.7252, PR-AUC=0.2188, positive predictions=0
Starting seed 18..
Completed seed 18: balanced accuracy=0.5000, ROC-AUC=0.6970, PR-AUC=0.2439, positive predictions=0
Starting seed 21..
Completed seed 21: balanced accuracy=0.5000, ROC-AUC=0.6830, PR-AUC=0.1565, positive predictions=0
Starting seed 24..
Completed seed 24: balanced accuracy=0.5000, ROC-AUC=0.7186, PR-AUC=0.1668, positive predictions=0
Starting seed 27..
Completed seed 27: balanced accuracy=0.5000, ROC-AUC=0.6563, PR-AUC=0.2024, positive predictions=0
Starting seed 30..
Completed seed 30: balanced accuracy=0.5000, ROC-AUC=0.6630, PR-AUC=0.1845, positive predictions=0
```

Figure 2.3.5: Results of the 10 baseline experiments

Figure 2.3.5 shows the results of the 10 repeated experiments using the original eLCS model. The seeds used were [3, 6, 9, 12, 15, 18, 21, 24, 27, 30], verifying that each experiment was unique. In every run, the seed affected both the shuffled split and the random state of the eLCS. Besides the seed, all of the hyperparameters were kept the same throughout all 10 experiments. We will use the same seeds in Part 4 for the improved eLCS model to make the comparison between data fair.

2.4 LCS parameter settings used

	Parameter	Value
0	N	1000
1	acc_sub	0.99
2	beta	0.2
3	chi	0.8
4	delta	0.1
5	discrete_attribute_limit	10
6	do_GA_subsumption	True
7	do_correct_set_subsumption	False
8	fitness_reduction	0.1
9	init_fit	0.01
10	learning_iterations	10000
11	match_for_missingness	False
12	mu	0.04
13	nu	5
14	p_spec	0.5
15	random_state	3
16	reboot_filename	None
17	selection_method	tournament
18	specified_attributes	[]
19	theta_GA	25
20	theta_del	20
21	theta_sel	0.5
22	theta_sub	20
23	track_accuracy_while_fit	False

Figure 2.4.1: Parameters List

When creating the baseline model, we used the seed as the random_state. So all **parameters were kept the same except for the seed** for each baseline experiment. The parameters in Figure 2.4.1 were retrieved using “model.get_params()”, which listed all of the parameters from the newly created eLCS object. Each of these affects how the experiment runs; for example, “learning_iterations” represents how many times eLCS performs learning iterations, “N” represents the maximum rule-population size, and “mu” represents the mutation probability. The rest of the explanations are in “Original_eLCS_Baseline.ipynb” in the project files.

```
# 10 reproducible seeds for the repeated experiments.
SEEDS = [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
✓ 0.0s
```

Figure 2.4.2: Seeds list

We used 10 different seeds (Figure 2.4.2) in **10 different and unrelated baseline experiments**. For each seed, a new eLCS model object was created, where the seed affected the train-test set split and the randomness of the experiment. The seed used for each experiment was recorded in order to make each run reproducible. As shown in Figure 2.3.5, the seed was recorded along with the experiment results for each run.

2.5 Baseline performance results

	Metric	mean	std	min	max
0	Accuracy	0.978985	1.170278e-16	0.978985	0.978985
1	Precision	0.000000	0.000000e+00	0.000000	0.000000
2	Recall	0.000000	0.000000e+00	0.000000	0.000000
3	F1	0.000000	0.000000e+00	0.000000	0.000000
4	Balanced accuracy	0.500000	0.000000e+00	0.500000	0.500000
5	ROC-AUC	0.686804	3.743284e-02	0.608623	0.735379
6	PR-AUC	0.182506	4.613999e-02	0.078235	0.243927
7	Positive predictions	0.000000	0.000000e+00	0.000000	0.000000
8	Training seconds	13.091345	1.869238e-01	12.765835	13.357953

Figure 2.5.1: Baseline eLCS results after runs

who actually did have a heart disease, the model was unable to identify any of them. This is extremely unsuccessful because the entire purpose for the model is to detect this target. Additionally, recall, and F1 score were all zero. Since there were no positive heart disease predictions, these scores cannot be calculated and are therefore zero. The balanced accuracy was 0.50 according to Figure 2.5.1. This means the model has chance-level performance, telling us that our model is the same as guessing. This is caused when the model only predicts one class and never the other.

The ROC-AUC and PR-AUC look at the probability scores. Some patients received higher scores, but because they weren't high enough, they weren't predicted as having heart disease. The 0.687 value of ROC-AUC means that the model can separate the two classes, but it is not strong enough to cause positive predictions. PR_AUC is more useful for this experiment because of the imbalanced dataset. The value is 0.183 for PR_AUC, which means that the model's probability scores ranked heart-disease cases better than random. Even though this checks the relationship between precision and recall, which were both zero, it only needs the scores, which is why PR_AUC isn't also zero.

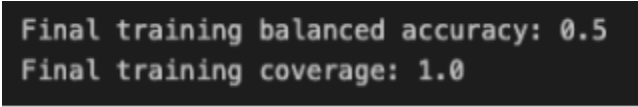
	Confusion category	mean	std	min	max
0	True negatives	55016.0	0.0	55016.0	55016.0
1	False positives	0.0	0.0	0.0	0.0
2	False negatives	1181.0	0.0	1181.0	1181.0
3	True positives	0.0	0.0	0.0	0.0

Figure 2.5.2: Confusion Category Summary

The accuracy from the baseline experiment was approximately 97.9%. However, this is only because many records belong to class 0. If the model predicts every record as class 0, then the final accuracy will be 97.9%. Due to this, the value from the result does not give us the true model accuracy we are looking for. The precision of the baseline results is 0, which means for everyone in the data set

According to Figure 2.5.2, all experiments correctly predicted 55,016 class 0 cases, and incorrectly predicted 0 class 1 cases. There were no false positives or true positives, meaning the model never

predicted class one, and that no heart disease cases were detected. Zero false positives is usually a good thing, but in this case, it didn't predict heart disease at all, meaning that it missed all the positive cases, as shown by the false negatives.



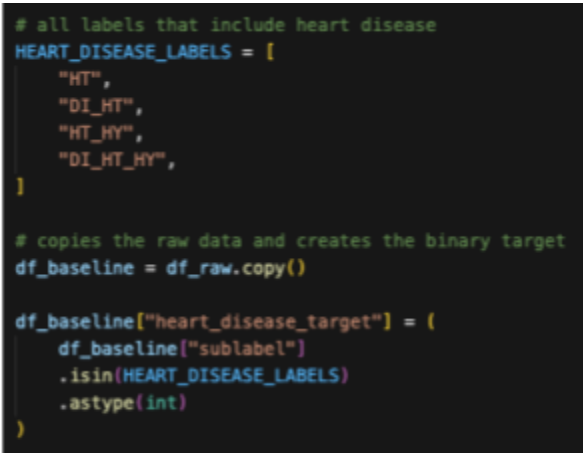
```
Final training balanced accuracy: 0.5
Final training coverage: 1.0
```

Figure 2.5.3: Final accuracy and coverage

From the experiment results shown in Figure 2.5.3, the final balanced accuracy was 0.5, and the final coverage was 1.0. A training balanced accuracy of 0.5 tells us that the model could not distinguish between the two classes in the

training data. Because only class 0 was predicted and no class 1 predictions occurred, the equation for balanced accuracy returns 0.5. This means that the model only learned class 0, which was “no heart disease”. The training coverage of 1.0 tells us that all of the training samples matched at least one baseline eLCS rule. However, most of the rules predicted class 0, meaning that the rules were not effective enough to predict the minority class being class 1.

2.6 Minimal processing required for LCS compatibility



```
# all labels that include heart disease
HEART_DISEASE_LABELS = [
    "HT",
    "DI_HT",
    "HT_HY",
    "DI_HT_HY",
]

# copies the raw data and creates the binary target
df_baseline = df_raw.copy()

df_baseline["heart_disease_target"] = (
    df_baseline["sublabel"]
    .isin(HEART_DISEASE_LABELS)
    .astype(int)
)
```

Figure 2.6.1: Target construction

Before running the dataset through the baseline eLCS model, we had to make it compatible. Before, there was no target column for the model to make predictions into, so we created the heart_disease_target column as the target. The sublabel attribute was used to create the target value, which contained combined keys such as HT, DI, HY, and N. HT represented heart disease, so any records with HT in their sublabel were mapped to class 1 (heart disease), and the other records that

didn't contain HT were mapped to class 0 (no heart disease). The true/false values were converted to numeric 1 and 0 because the eLCS model requires numeric values. This was necessary in order for the model to use this dataset, and the original dataset was left untouched for the sake of backup.

```
# the target-related columns that cannot be predictors
EXCLUDED_COLUMNS = [
    "heart_disease_target",
    "sublabel",
    "disease_flags",
    "heart_disease",
]

# separates the predictor variables from the target
X = df_baseline.drop(columns=EXCLUDED_COLUMNS)
y = df_baseline["heart_disease_target"].astype(int)
```

Figure 2.6.2: Excluded Columns

```
# create a shuffled and stratified 80/20 train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=SEED,
    stratify=y,
    shuffle=True,
)
```

Figure 2.6.3: 80/20 split

Figure 2.6.2 shows how `heart_disease_target`, `sublabel`, `disease_flags`, and `heart_disease` were excluded from being used as predictors. `Heart_disease_target` was separated as the target, so it can't be the predictor. `Sublabel` was used to construct the target, so this also could not be used, as it would leak the answer. `Disease_flags` held information about disease, which could also leak the answer. Finally, `heart_disease` held information about the patient that the eLCS model could use to ignore every other value and guess whether the patient had heart disease or not. The 'label' column indicates if a patient is 'Normal' or 'Abnormal'. Although it does have a correlation with the target, it is very broad, and doesn't directly have a one to one correlation with heart disease as it could indicate different types of disease. Because of this, we decided to not remove this column from the baseline to keep the processing as minimal as possible.

Figure 2.6.3 shows how the 80/20 split was used to create the training and testing sets. Shuffling was used in order to reduce the influence of the original dataset's row order. The `test_size=20` makes sure that the test set is 20% of the records and the train set will be 80% of the records. Stratification was also used so that the 2.1% class 1 rate would remain in both sets; this is to keep both sets as similar to the original dataset as possible. Finally, a random seed was used so that this split could be reproducible.

```
# creates an encoder that can handle unseen test-set categories
encoder = OrdinalEncoder(
    handle_unknown="use_encoded_value",
    unknown_value=-1,
)

# copies the test/train sets before replacing categorical columns
X_train_encoded = X_train.copy()
X_test_encoded = X_test.copy()

# learns category mappings from the training data and applies them to both partitions.
X_train_encoded[categorical_columns] = encoder.fit_transform(
    X_train[categorical_columns]
)
X_test_encoded[categorical_columns] = encoder.transform(
    X_test[categorical_columns]
)
```

Figure 2.6.4: Category encoding

The categorical attributes in the dataset were converted into numerical values so that they could be processed through the eLCS model. Each categorical value was given a numerical code, and the encoder learned these mappings and applied it to the testing set. Using `fit_transform()` and `transform()` on the training and testing sets, respectively, prevented data leakage.

3. Data Preprocessing and Feature Engineering

3.1 Data leakage prevention measures

To prevent target leakage of `heart_disease_target`, there are multiple decisions made during the processing phase to address this issue. Two measures have been used to reduce the data leakage, Train/test leakage and target leakage.

```
Training shape: (222960, 39)
Testing shape: (55741, 39)

Training target distribution:
heart_disease_target
0    218247
1     4713
Name: count, dtype: int64

Testing target distribution:
heart_disease_target
0    54563
1     1178
Name: count, dtype: int64

Training positive rate:
2.1138%

Testing positive rate:
2.1133%
```

Figure 3.1.1: Train/Test leakage

The train/test split has been used before any other preprocessing measures. This is because splitting the data used for training and model testing should be divided. Models that are based on the same instances of data would cause the model to practically memorise the pattern instead of learning new patterns. A 80/20% split is used when 80% of the data is used for training while 20% is used for testing (The 80/20 split would be further explained in **section 5**). Additionally, splitting the testing and training data stops the testing data from influencing the preprocessing phase, e.g. calculating the median of glucose, as the testing data would change the median compared to just using the training data.

```
Removed target-leakage columns: ['sublabel', 'disease_flags', 'heart_disease', 'label']
Training shape: (222960, 35)
Testing shape: (55741, 35)
```

Figure 3.1.2: Data leakage

These columns label, sublabel, disease_flags, and heart_disease are removed in the processing stage because they have partial or full information about the target. In task 2, we did not remove the 'label' column because it was not a one to one correlation with the target, however it still does classify an individual as abnormal or normal with any type of disease, including heart disease so it will need to be addressed properly during preprocessing and removed. For the sublabel column, this was used to directly create the target because it contains the classification of diseases. The Disease_flags column contains the binary indicator of which disease the individual has. Lastly, the heart_disease column shows the probability of heart disease for a patient with similar traits and showed a strong correlation of 0.65 with the target; therefore, it cannot be used.

3.2 Data cleaning

In phase 1, the raw dataset has been cleaned for exploratory analysis (including missing values, duplicate records, inconsistent categories, and data types). Before going further, the dataset has been reviewed again to check if there are any issues after the splitting of the training and testing datasets. Invalid values were partially checked during phase 1 EDA; therefore, we will review them again.

We checked the missing values in the training and testing sets. No missing values were found. This is to ensure that the cleaning completed in phase 1 was inspected again.

```
Dataset shape: (278701, 40)
Missing values: 0
Exact duplicate records: 0

Target values:
[np.int8(0), np.int8(1)]

Target distribution:
heart_disease_target
0      272810
1       5891
Name: count, dtype: int64
```

3.2.1: Missing and duplicate records

When checking for the amount of duplication rows in the training and testing dataset after the removal of data leakage, showed that there are 25,812 instances in the training dataset and 2078 instances in the testing dataset that has at least 1 other instances with the same input feature values. The Figure 3.2.2 also shows the total number of repeated different combinations of input features that the training dataset has (11,285). Out of these 707 groups conflicting information about different instances with the same input features having different target values.

This is because matching predictor values do not inherently mean that the duplicate values are the same instance or is invalid, unlike during phase 1 where the identified duplicate rows were identical throughout the 39 columns which were removed in the cleaning process. The removal of all of these instances could result in discarding valid information as well as since our minority class (Heart disease) is already small, reducing these would worsen class imbalance.

Invalid values (Figure 3.2.3) were checked by making sure that the values are within valid boundaries, this include values that are negative and check that the values are within clinical measurements. Age, BMI, glucose, cholesterol, systolic_bp, diastolic_bp, and heart_rate all showed that the values are plausible.

```
composite_key:
composite_key
Middle_Female_Overweight_Never_No 4441
Mature_Female_Overweight_Never_No 4302
Mature_Female_Obese_Never_No 4029
Middle_Female_Obese_Never_No 4011
Middle_Female_Normal_Never_No 3853
...
Young_Female_Underweight_Current_No 148
Young_Female_Underweight_Never_Yes 148
Young_Male_Underweight_Former_Yes 138
Young_Male_Underweight_Current_Yes 133
Young_Female_Underweight_Former_Yes 119
Name: count, Length: 192, dtype: int64

age_level:
age_level
Mature 69654
Middle 67574
Senior 55874
Young 29858
Name: count, dtype: int64

gender:
...
hypertension 139929
diabetes 75162
heart 7869
Name: count, dtype: int64
Output is truncated. View as a scrollable element or open in a tool
```

3.2.4: Inconsistent categories

```
composite_key      str
age_level          str
gender            str
bmi_level          str
smoking           str
diabetes          str
age               int64
age_normalized     float64
bmi               float64
hypertension       float64
HbA1c_level       float64
glucose           float64
cholesterol        float64
sleep_hours       float64
triglycerides     float64
physical_activity  str
family_history     str
stress_level       str
low_hdl_cholesterol str
high_ldl_cholesterol str
blood_pressure    float64
high_blood_pressure str
sugar_consumption str
crp_level         float64
homocysteine_level float64
...
education_level   str
employment_status str
source_dataset    str
dtype: object
```

Figure 3.2.5: Data Types

All categorical columns were checked for inconsistent categories. Composite key, age_level, gender, bmi_level, smoking, diabetes, physical_activity, family_history, stress_level, low_hdl_cholesterol, high_ldl_cholesterol, high_blood_pressure, sugar_consumption, education_level, employment_status and source_dataset, all have consistent categorical values and formatting consistency.

The data types of each column are correct; therefore, no changes would be made (Figure 3.2.5).

3.3 Outlier investigation and handling

	feature	minimum	maximum	lqr_lower_bound	lqr_upper_bound	flagged_records
0	age	2.000000	89.000000	-22.000000	122.000000	0
1	hypertension	0.000000	1.000000	-0.325594	0.542657	6454
2	HbA1c_level	3.500000	9.000000	3.170101	9.083164	0
3	glucose	70.000000	300.000000	10.000000	250.000000	1624
4	cholesterol	150.000000	300.000000	138.000000	306.000000	0
5	sleep_hours	4.000000	10.000000	3.950000	9.950000	1216
6	triglycerides	50.000000	400.000000	-42.625000	412.375000	0
7	blood_pressure	120.000000	180.000000	146.284659	153.382711	24303
8	crg_level	0.003647	14.996625	6.599316	8.454110	24155
9	homocysteine_level	5.000236	19.999037	11.690145	13.283792	24774
10	systolic_bp	90.000000	179.000000	96.500000	172.500000	21854
11	diastolic_bp	60.000000	119.000000	63.500000	115.500000	18579
12	alcohol_intake	0.000000	30.000000	2.750000	27.150000	26112
13	salt_intake	2.000000	15.000000	3.300000	13.700000	26854
14	heart_rate	50.000000	99.000000	52.500000	96.500000	16665
15	hdl	30.000000	99.000000	37.000000	93.000000	25988
16	ldl	70.000000	189.000000	79.500000	179.500000	23413

Figure 3.3.1: lower, upper, outliers

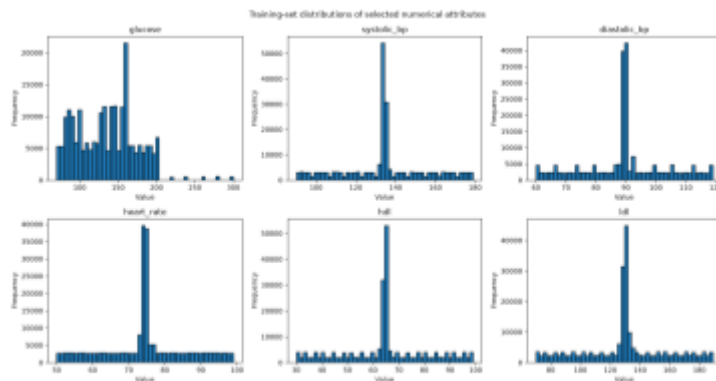


Figure 3.3.2: Outlier investigation

In phase 1, BMI was found to have a considerable number of outliers, but is proven that the outliers shouldn't be transformed or removed. The other columns were discussed at a surface level, based on the boxplot, and showed no isolated instances. To further investigate this, the Interquartile range method was performed on the numerical columns. Triglyceride, cholesterol, and age showed no outliers, but the rest of the columns' results displayed incredibly high values, e.g 18579 outliers for diastolic_bp as well as other columns. A histogram is created to show the distribution of instances across each column. Most of the columns (systolic_bp, diastolic_bp, heart_rate, hdl, ldl, glucose) showed a high peak in the middle and consistent values across the whole range. The IQR method identified

many statistical outliers, however this does not automatically make them invalid. Inspecting the histograms and range values, they did not provide enough evidence to justify that these identified outliers would be treated as valid instances. Thus, no removal, transformation, or capping is performed.

3.4 Feature engineering

```
Removed redundant and provenance columns: ['composite_key', 'source_dataset', 'age_level', 'age_normalized', 'bmi']  
Training shape: (222960, 30)  
Testing shape: (55741, 30)
```

Figure 3.4.1: Removal of redundant and provenance variables

In the current dataset up to this point of processing, multiple columns are irrelevant or provide no value towards predicting the target, these include:

Composite key (removed): A composite key is just the combination of variables (age_level, gender, bmi_level, smoking, and diabetes); therefore, this does not provide any value, as it is simply grouping the variables for instance. So removing this would not cause any information to be lost and is just an unnecessary column to provide to the model.

Age_level and age_normalized (removed): Age_level and age_normalized provide similar information to age, but age_level is a categorical level and age_normalized is the linear scaling of age and showed a 1.00 correlation with age during EDA. These add no additional information about age, as age could be discretised independently for eLCS, thus being a more favourable column to remain in the training dataset.

bmi (removed): The bmi column is removed because the bmi_level column follows the WHO standard, which is the standard classifier for BMI. The bmi column does allow independent interpretation of discretisation, but since bmi_level is already in a suitable standard, there isn't any reason not to use bmi_level instead of bmi.

Source_dataset (removed): The source dataset has been removed because the dataset "281K Patient Dataset for Multi-Disease Prediction" is from the combination of three separate dataset. Each dataset varies in the proportions of instances with heart disease, therefore the model could use this to learn the pattern of which dataset is more likely to have heart disease resulting in its influence on its prediction.

3.5 Encoding categorical variables

	gender	diabetes	family_history	low_hdl_cholesterol	high_hdl_cholesterol	high_blood_pressure
226737	1	1	0	0	0	0
240229	0	0	1	0	1	0
263800	1	0	0	0	0	1
102871	1	1	1	0	1	0
209771	0	1	0	0	0	1

	bmi_level	physical_activity	stress_level	sugar_consumption	education_level
226737	3	1	0	1	1
240229	0	0	1	2	0
263800	2	0	1	2	0
102871	3	1	2	2	2
209771	0	1	1	1	0

	smoking_Current	smoking_Pastor	smoking_Never	employment_status_Employed	employment_status_Retired	employment_status_Unemployed
226737	0	0	1	0	0	1
240229	0	1	0	0	0	1
263800	1	0	0	0	1	0
102871	1	0	0	0	0	1
209771	0	0	1	0	0	1

Figure 3.5.1: Encoded columns

All the categorical variables have been encoded to numerical values because eLCS could not accept values in strings. For example, gender has been changed from “Female” and “Male” to 0 and 1.

Multiple types of encoding have been used for the categorical variables since some variables required different encoding methods.

Binary Encoded

For these columns (gender, diabetes, family_history, low_hdl_cholesterol, high_hdl_cholesterol, and high_blood_pressure), binary encoding is used because these columns only have 2 possible outputs.

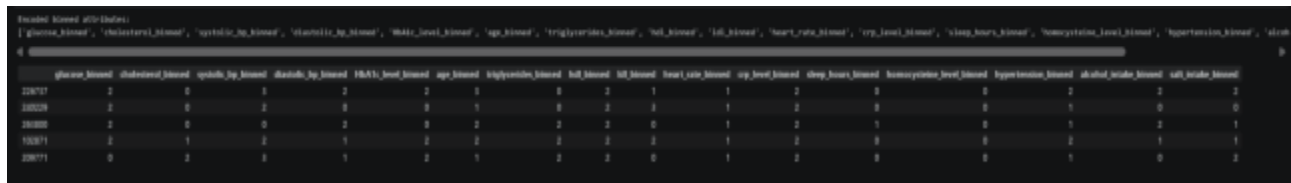
Ordinal Encoded

For these columns (bmi_level, physical_activity, stress_level, sugar_consumption, ”education_level”), ordinal encoding is used because, for example, for stress_level, there is an order of the output values “low”, “medium,” and “high”; therefore, the encoding converts it into numerical values (1 to 3).

One-Hot Encoded

For the rest of the categorical columns (“smoking”, ”employment_status”), one-hot encoding is used since they provide no order and cannot be represented in binary; therefore, new columns would be created to represent each possible outcome, as shown in Figure 3.51 .

3.6 Discretising & Encoding numerical variables



	glucose_bin	cholesterol_bin	systolic_bp_bin	diastolic_bp_bin	hba1c_bin	age_bin	triglycerides_bin	hdl_bin	ldl_bin	heart_rate_bin	rpg_bin	sleep_hours_bin	hemocritbin_bin	hypertension_bin	alcohol_bin	salt_bin
228707	2	0	0	0	2	0	0	2	1	1	2	0	0	2	0	2
100029	2	0	2	0	0	1	0	2	2	1	2	0	0	1	0	0
100000	2	0	0	0	2	2	2	2	0	1	2	1	0	1	2	1
100071	0	1	2	1	2	2	2	2	2	1	2	0	0	2	1	1
200771	0	2	0	1	2	1	0	2	0	1	2	0	0	1	0	2

Figure 3.6.1 Discretise and Encode the numerical attribute

Discretization has been used for all the attributes in Figure 3.6.1, changing the numerical dataset to categorical values because it simplifies feature representation as well as making it easier to interpret the rule. Even though eLCS is able to utilize the ranges on a classifier, removing the number of grouping would reduce the number of possible distinct values thus may make the rule learning much easier. The downside is the loss of detail in the data which would make the rule more generalized and could affect eLCS predictive performance. To determine the categorical values, the columns that have a clinical standard include a fixed-boundary, the boundaries set are low, medium or high each with their own range for each categorical value. For example, for glucose, the normal range would be below or at 100mg/dL, the prediabetes level from 100 to 126mg/dL and diabetes at 126mg/dL and above.

The columns (hypertension, alcohol_intake and salt_intake) that don't have a set standard for measuring different levels are based on training derived quantile based boundaries. This is to roughly split training instances across 3 categorical levels (3 bins) based on the quantile percentage that each instance resides in.

These attributes are then encoded using ordinal encoding as the categorical values are arranged in order. The encoding is applied on both the training and testing dataset as well as the original attributes are removed and have been replaced by the created bins.

3.7 Class-imbalance handling

In the EDA, the target variable is highly imbalanced across the dataset, with 2.1% of instances with heart disease and 97.9% of instances without heart disease of the total 278,701 instances. The baseline eLCS prediction results showed that the final balanced accuracy test is 0.5. This balanced accuracy occurred because the baseline model practically predicted only the class which has no disease, and failed to identify any with a heart disease. This shows that the model does not provide any useful balanced predictions between the two classes.

The imbalance would be handled after receiving the result from the prediction preprocessed dataset (*Task 4*) in eLCS. This is because it would be valuable insight into how much class imbalance could affect the overall prediction of the model when the imbalance is handled.



Figure 4. Distribution of the heart disease target in the cleaned dataset from phase 1

3.8 Expected impact of preprocessing on LCS performance

The overall preprocessing phase of the dataset is expected to improve the eLCS ability to process and performance, these improvements include:

- **Improved Computational cost / quicker prediction:** In the feature selection process, multiple redundant columns were removed, therefore there are less combinations of if and then rules.
- **More compact rule set:** Applying the discretization lowers the amount of classifiers since there will be less classifiers that are tied to continuous values which also increase readability for validation and inspection of the rule.
- **Improved accuracy of rules:** Using the appropriate encoding types would reduce the number of misleading information when just applying a general encoding type to all columns which causes the eLCS to learn inaccurate patterns.
- **Increase trustworthiness of prediction:** This is because the columns that are removed that provide information about the target information would influence eLCS ability to learn and accurately predict the target without genuinely learning from the other features.

4. Improved LCS-Based System

4.1 Summary of changes vs. the original LCS system

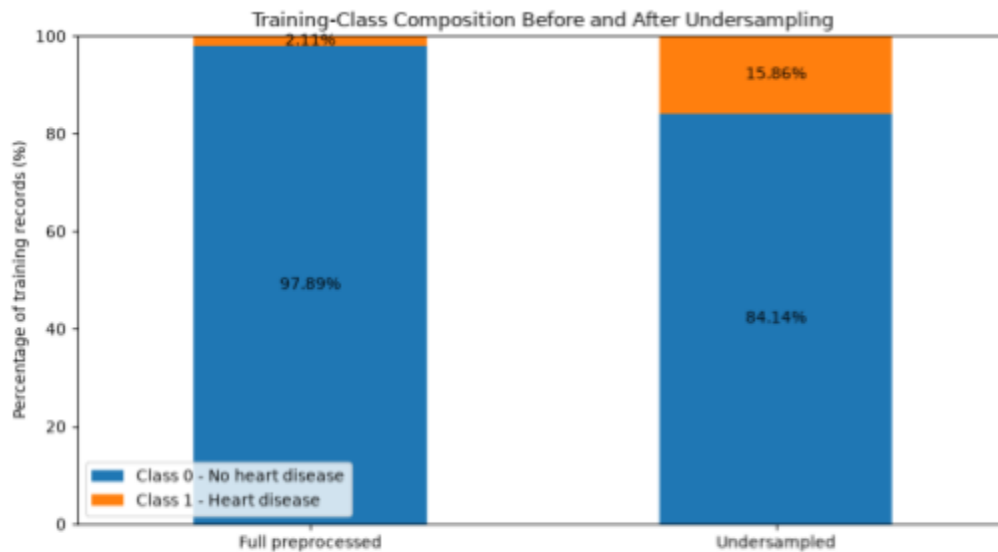


Figure 4.1.1: Training distributions before and after undersampling

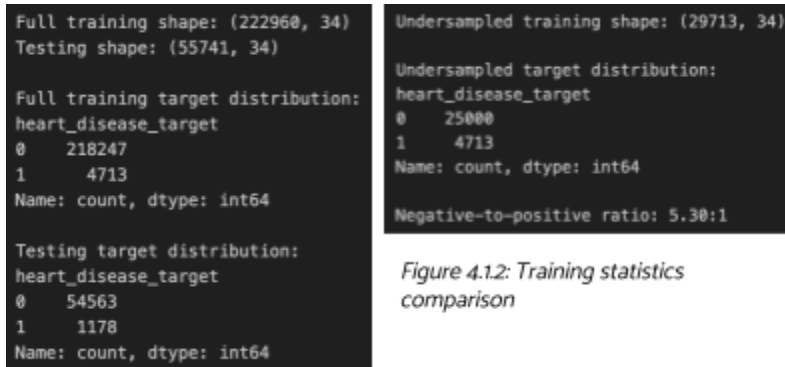


Figure 4.1.2: Training statistics comparison

This section focused on improving the predictions of the eLCS model by using undersampling and tuning the hyperparameters. The full preprocessed data information is displayed in Figure 4.1.1. There were a total of 222,960 records in

the training set, where 218,247 of them were class 0, and 4,713 of them were class 1. Class 0 significantly outnumbers class 1 in the total number of records, which results in a class imbalance problem. This directly correlates with another issue, which was the model hyperparameters not fitting the dataset. The default population and learning_iteration values were not making use of all the data provided and didn't give the model enough time for training.

To address the class imbalance issue, sampling was used to randomly select 25,000 class 0 records to keep in the training set. The rest of the class 0 records were removed from the training set, resulting in a total training set size of 29,713 records. This resulted in a better ratio of 5.3 class 0 records for every class 1 record. Before changing the hyperparameters, the new undersampled training set was used to train the model, just to see what different undersampling did by itself.

	Seed	Accuracy	Precision	Recall	F1	Balanced accuracy	ROC-AUC	PR-AUC	True negatives	False positives	False negatives	True positives	Positive predictions	Training seconds
0	3	0.978723	0.100000	0.000849	0.001684	0.500342	0.601036	0.061937	54554	9	1177	1	10	11.185162
1	6	0.978867	0.000000	0.000000	0.000000	0.500000	0.579673	0.031564	54563	0	1178	0	0	11.300214
2	9	0.978867	0.000000	0.000000	0.000000	0.500000	0.489641	0.022186	54563	0	1178	0	0	11.221245
3	12	0.978867	0.000000	0.000000	0.000000	0.500000	0.551384	0.040087	54563	0	1178	0	0	12.375019
4	15	0.978867	0.000000	0.000000	0.000000	0.500000	0.599107	0.055318	54563	0	1178	0	0	11.700800
5	18	0.978867	0.000000	0.000000	0.000000	0.500000	0.437362	0.025779	54563	0	1178	0	0	11.522431
6	21	0.978867	0.000000	0.000000	0.000000	0.500000	0.583367	0.046269	54563	0	1178	0	0	11.965966
7	24	0.978597	0.222222	0.006093	0.009959	0.502354	0.571226	0.040622	54542	21	1172	6	27	11.656360
8	27	0.978867	0.000000	0.000000	0.000000	0.500000	0.570092	0.035616	54563	0	1178	0	0	11.256830
9	30	0.978867	0.000000	0.000000	0.000000	0.500000	0.531064	0.026285	54563	0	1178	0	0	11.527865

	Seed	Accuracy	Precision	Recall	F1	Balanced accuracy	ROC-AUC	PR-AUC	True negatives	False positives	False negatives	True positives	Positive predictions	Training seconds
0	3	0.977593	0.211382	0.022071	0.039969	0.510147	0.795535	0.112511	54466	97	1152	26	123	5.231066
1	6	0.971888	0.194682	0.105263	0.136639	0.547931	0.809791	0.113939	54050	513	1054	124	637	5.396431
2	9	0.976373	0.216327	0.044992	0.074491	0.520736	0.754192	0.099405	54371	192	1125	53	245	6.183733
3	12	0.967241	0.183594	0.159593	0.170754	0.572135	0.734273	0.104268	53727	836	990	188	1024	5.223253
4	15	0.977073	0.191358	0.026316	0.046269	0.511957	0.755534	0.084237	54432	131	1147	31	162	5.083849
5	18	0.974794	0.171014	0.050085	0.077479	0.522422	0.806915	0.112479	54277	286	1119	59	345	5.426928
6	21	0.976803	0.239819	0.044992	0.075768	0.520956	0.753362	0.089727	54395	168	1125	53	221	5.109017
7	24	0.978813	0.200000	0.000849	0.001691	0.500388	0.752372	0.103252	54559	4	1177	1	5	5.391051
8	27	0.978526	0.256410	0.008489	0.016434	0.503979	0.796370	0.113459	54534	29	1168	10	39	5.572949
9	30	0.976463	0.203540	0.039049	0.065527	0.517875	0.795410	0.082736	54383	180	1132	46	226	5.360582

Figure 4.1.2: Preprocessing vs Undersampling results

Figure 4.1.2 shows the difference between using the fully preprocessed and undersampled training set. The undersampled training experiment produced much better results, because the default eLCS model was able to predict many more true positives in all 10 experiments. False negatives are much more critical in healthcare, because it means deadly diseases go unnoticed until it's too late. Fewer false negatives and more true positives in the undersampled experiments were a sign that it produced better results than the preprocessed experiments.

Configuration	Training data	N	Learning iterations
0 Default eLCS	Full or undersampled	1000	10000
1 Improved eLCS	Undersampled	8000	50000

Figure 4.1.3: Improving the eLCS model

After checking that undersampling succeeded, we had to change the eLCS hyperparameters so that it fit the needs of the undersampled

training set. N represents the population of the rules, which by default is 1000 in eLCS.

This is very little for our dataset and restricts the model's learning capacity. Learning iterations is how many records the model will go through in an experiment, and 10,000 iterations means that it will go through 10,000 records in each experiment. This is a significant limitation for our training set, because it means it will only go through $\frac{1}{3}$ of the training set in each experiment (skipping most of the training set). To overcome this limitation, the learning iterations were increased from 10,000 to 50,000 so that it would go over every record at least once in each experiment. Increasing it could improve the experiment results, but would also increase computational costs. Everyday laptops aren't made for machine learning, so we decided to limit the learning iterations to 50,000. There was no modification of the eLCS source code, only the hyperparameters.

There were many variables that did not change throughout the different experiments. The first one was the test set, which was kept the same with 55,741 records (54,563 class 0 records and 1,178 class 1 records). The same seeds were used throughout every 10 repeated experiments in tasks 2 - 4 to keep the experiments reproducible and not introduce any biases. Separating these changes into 3 parts allowed us to understand how each change affected the final predictions made by the eLCS model.

4.2 Justification

Sampling was used to solve the class imbalance issues in our dataset. The full training dataset contained approximately 46 negative records for every single positive record. This led to the eLCS systems predominantly learning only the class 0 rules. At first, the high accuracy of 97.9% was interpreted as good; however, this is misleading because predicting class 0 for everything gives a 97.9% accuracy. Reducing the class 0 records gave class 1 records more influence during the rule discovery process. Class 1 records were all retained because we could not afford to lose any real heart-disease examples. The other solution was to generate new class 1 records, but not only would artificial records be bad for the model training, but it would also increase the computational cost.

Changing the N hyperparameter to 8,000 allowed the model's population to contain more candidate rules. The purpose of increasing the population was to give the model more capacity to be able to retain classifiers/patterns that would otherwise be competing with each other in a smaller population. Changing learning iterations to 50,000 gives rules more opportunities during the experiments to be matched together, evaluated, reproduced, and refined. Increasing the iterations also

means that the model is able to properly go through and learn from all the training instances at least once. While changing the N and learning iterations hyperparameters resulted in better results, it also cost us triple the training time, from 30 minutes to approximately 90 minutes.

4.3 Pseudocode of the improved system

```
Split the cleaned data into stratified training and testing sets

Fit preprocessing transformations on the training data
Apply the fitted transformations to the test data

Save the fully preprocessed training data

positive_records = all class 1 training records
negative_records = randomly selected sample of 25,000 class 0 records
undersampled_training = shuffle(positive_records + negative_records)

For each seed in [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
    Model A:
        Train default eLCS model on fully preprocessed training data
    Model B:
        Train default eLCS model on undersampled training data
    Model C:
        Train improved eLCS (N=8000, Iter=50000) on undersampled
        training data.

Evaluate all models on the same test set (remains unchanged)
Store the metrics, confusion matrix, and training time
```

Figure 4.3.1: Explanation of the improved system in Pseudocode

Figure 4.3.1 describes how the improved eLCS system works in a simplified way. It starts by using the phase 1 dataset and divides it into stratified training and testing sets with an 80/20 split. The preprocessing is learned from the training data and then applied to the testing set, while preventing test-data leakage. All class 1 records are kept, while 25,000 randomly selected class 0 records are retained. There are three different configurations of models that allow each change to be inspected individually. All models use the same test set, which was kept near the beginning of the code, and are compared, with the results being stored in files.

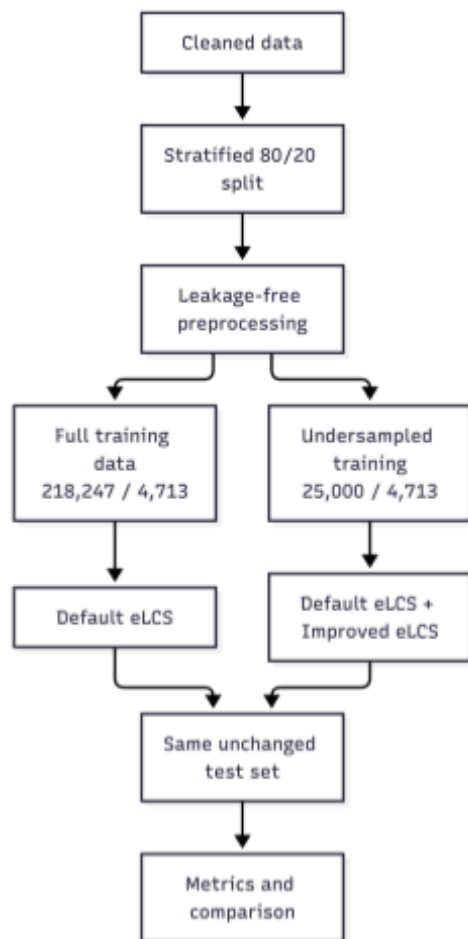


Figure 4.3.2: Improved eLCS experimental pipeline

This flowchart in Figure 4.3.2 gives a visual overview of how the data moves through the improved eLCS pipeline. The split that happens after leakage-free preprocessing demonstrates how data is split into two groups, which are the “full-training” branch and the “undersampled” branch. The left branch works with the data that hasn’t been through imbalance treatment. The right branch works with data that goes through imbalance treatment and through both the baseline and improved eLCS models. The convergence of both branches visually demonstrates how they both use the same testing set so that no model has an advantage over the other.

This entire pipeline is reproducible due to all information being recorded and saved, like the 80/20 split information, the sampling process,

and the same 10 eLCS seeds that are used during the repeated tests. This flow chart is a visual version of the pseudocode from Figure 4.3.1, which explains how the system works in a simplified way.

4.4 Comparison against the original LCS baseline

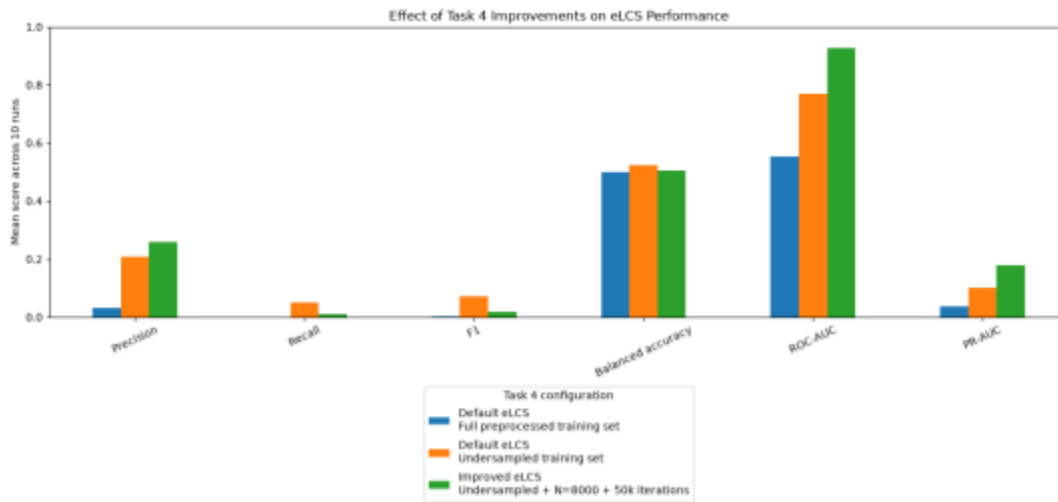


Figure 4.4.1: Comparison of eLCS model

In Figure 4.4.1, each bar represents the mean score across 10 seeds. To make the comparison fair and remove any possible biases, the same test set was used across all three scenarios. The accuracy was left out because the 97.9% majority class makes it misleading. Therefore, the six variables in the graph were used to evaluate the differences between the different scenarios.

Blue represents the **fully preprocessed training set with the default eLCS model**. Precision was only 0.032, and both recall and F1 were almost zero at 0.0006 and 0.0012, respectively. Finally, the balanced accuracy was 0.500. This means that preprocessing alone is not enough for the model to be able to predict heart disease (class 1). In the 10 repeated experiments for this scenario, the model kept predicting everything as class 0, with an average of less than one class 1 prediction per run.

Orange represents the **undersampling training set with the default eLCS model**. This was where visible differences appeared in the results, with precision increasing from 0.032 to 0.207, recall increasing from 0.0006 to 0.050, F1 increasing from 0.0012 to 0.071, and balanced accuracy increasing from 0.500 to 0.523. The ROC-AUC and PR-AUC values saw significant improvements, from 0.551 to 0.769 and from 0.038 to 0.102, respectively. Overall, this scenario achieved the highest recall, F1 and balanced accuracy scores out of all three experiments. These results suggest that sampling greatly improved the model's ability to predict positive cases.

Green represents the **undersampling training set with the improved eLCS model**. The same training set was used as in the previous scenario, but the model's parameters were tweaked to match the training set's needs. It achieved the highest precision at 0.260, the highest ROC-AUC at 0.926, and the highest PR-AUC at 0.178. The higher ROC-AUC value proves that, in this experiment, the improved model was much better at ranking heart disease cases over non-heart disease cases. The higher PR-AUC is the more important variable because our dataset's issue is class imbalance. It being higher proves that the improvement of ROC-AUC carried over to the minority class 1.

From the evaluation of the graph in Figure 4.4.1, we can safely assume that undersampling provided the greatest improvement in the model's ability to predict heart disease. Increasing the population and learning iterations of the model provided the greatest improvement in ROC-AUC and PR-AUC. Because no experiment was the best at everything, we should assume that the result was a mixed improvement from both sampling and tuning the eLCS model.

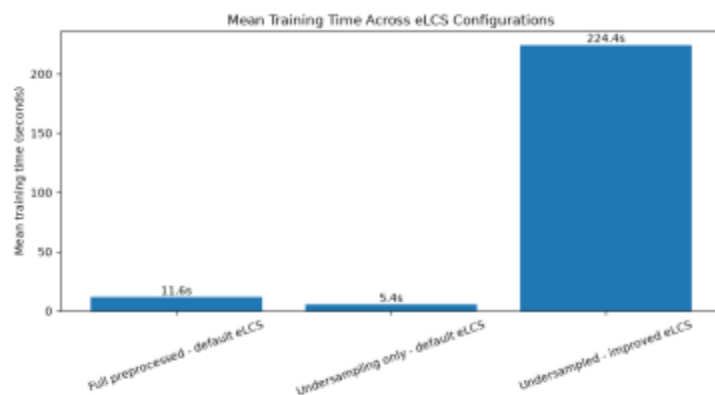


Figure 4.4.2: Comparison of experiment training time

In the first experiment, the mean training time was around **11.6 seconds**. This used all of the preprocessed training records, and while using all of the training data, it predicted almost zero heart disease cases. The second experiment's mean training time dropped to around **5.4 seconds**. This was due to the training set being reduced from 222,960 records to

29,173 records, which is approximately an 87% reduction in records. This reduced the computational cost of the experiment, causing the model to train faster and producing better recall, F1, and balanced accuracy scores. The third experiment's mean training time increased to around 224 seconds, which is approximately 42 times slower than the second experiment. This was mainly caused by the increase in population from 1,000 to 8,000 rules, and learning iterations from 10,000 to 50,000. There was a significant increase in computation cost, but it produced much higher ROC-AUC and PR-AUC values, signalling that the increased cost came with benefits. Whether the improvement in results justifies the greater computational cost depends on the situation. We needed better results from the model, which justified spending more time on the actual model training.

5. Experiments

5.1 Validation strategy, Justification of using 80/20 split over k-fold CV

We went with an 80/20 split, which split the dataset into a training set with 80% of the data and a testing set with 20% of the data. In the original dataset, approximately 2.1% of patients had heart disease. This is a very low number and introduces class imbalance to the dataset. To maintain this environment, stratification was used to keep the same 2.1% proportion, making the training and testing sets similar to the raw dataset. A seed was recorded and used for the split so that the 80/20 split could be reproducible, just in case we wanted to re-run the split again with the same results. The exact same seed was used in the baseline model to make sure the comparison between the baseline eLCS model and the improved model was fair. The experiment was repeated using 10 recorded random seeds so that the results were not based on a single train-test split.



Figure 5.1 Class Distribution After Stratified 80/20 Split

The 80/20 split was a better approach than k-fold cross-validation, mainly because of the size of the dataset. And the computational requirements. K-fold CV requires the model to be trained multiple times, with K representing the number of times the model will be trained. The eLCS model keeps on creating, evaluating, and evolving rules in each training, so this whole process happening repeatedly is very computationally costly. The dataset contains over 200k rows, which is more than enough for an 80/20 split to be effective for machine learning. If the dataset were smaller, then it would be less reliable because the training set would not contain enough data to train the model.

lower-upper bound, prediction array, crossover mutation, 7-8k population, 50,000 iterations, 25,000 majority class instances, match set, covering

5.2 Evaluation of metrics **DO NOT TOUCH >:CCCCC**

Because the dataset is highly imbalanced, with only around 2.1% of records belonging to the positive heart-disease class, accuracy alone is not sufficient to evaluate model performance. The main metrics selected for this project are **recall, PR-AUC, and balanced accuracy**, with **precision, F1-score, ROC-AUC, accuracy, and the confusion matrix** used as supporting measures.

Recall measures the percentage of actual positive cases that the model correctly identifies. In the context of this project, the recall tells us that out of all patients who have a heart disease, how many of them were correctly identified. Recall is especially important in our context because the main objective of our model is to identify true positive cases. Recall has been given more importance for this project because the main objective is to identify positive heart disease cases. A false positive is when a patient is classified as having a heart disease even though they don't, however this scenario is more desirable than if the model were to miss an actual positive case (false negative)

Precision measures the percentage of all the positive cases identified by the model being actual positives. In the context of this project, the precision tells us how many of the patients that were identified with heart diseases genuinely do have a heart disease. Precision is important because a model that predicts too many false positives would be less practicable and lose credibility. However we are treating precision not as important as recall for this project because being able to identify more actual heart disease cases is more important, even when with the compromise of more false positives.

The **F1 score** combines the recall and precision into one measurement. This is useful for this project because it has the desired effect of evaluating if the model can correctly identify positive cases, however at the same time considering false positives. The F1 score is less affected by the large imbalance of negative cases which makes this a much more suitable metric for this highly imbalanced classification problem. However, recall is still placed higher for this project because F1 gives precision and recall equal importance, however this project aims to identify as many heart disease cases as possible.

Balanced accuracy calculates the average performance across the positive and negative classes. This is particularly useful for the highly imbalanced dataset because both classes contribute equally to the score. In comparison, normal accuracy can remain very high simply because the majority of records belong to the negative class.

PR-AUC evaluates the relationship between precision and recall across different classification thresholds. It is one of the main metrics used in this project because it focuses strongly on performance for the minority positive class and is therefore well suited to an imbalanced classification problem. In the implementation, PR-AUC is calculated using the average precision score.

ROC-AUC measures how well the model separates positive and negative cases across different classification thresholds. It is useful for evaluating the overall ranking ability of the model, although it is treated as a supporting metric because strong ROC-AUC performance does not necessarily mean that the model identifies many positive cases at its final classification threshold.

Accuracy measures the percentage that all the predictions made by the model are correct. The accuracy has been reported for each of the models we ran, however it is not used as the main evaluation metric because our dataset is heavily imbalanced. Since 97.9% of the records fall into class 0 (no heart disease), a model which just predicts class 0 for every single instance could achieve an accuracy of 97.9% but still fails to identify even a single heart disease case. This is why for our case, the accuracy is not the main evaluation metric, but instead interpreted alongside other evaluation metrics that are better designed for class unbalances.

The **confusion matrix** reports the numbers of true positives, true negatives, false positives, and false negatives. It is used alongside the numerical metrics to show the actual types of prediction errors produced by the model, particularly the number of positive heart-disease cases that are missed.

Improved eLCS Prediction Outcomes

Actual class	Predicted class	
	Class 0 No heart disease	Class 1 Heart disease
Class 0 No heart disease	True Negative 99.94% Mean: 54,529.0	False Positive 0.06% Mean: 34.0
Class 1 Heart disease	False Negative 99.07% Mean: 1,167.0	True Positive 0.93% Mean: 11.0

Overall, recall, **PR-AUC**, and **balanced accuracy** are the **primary evaluation metrics** used in this **project**. Recall reflects the main objective of detecting actual heart-disease cases, PR-AUC evaluates minority-class performance across thresholds, and balanced accuracy measures performance across both classes without being dominated by the majority class. The remaining metrics provide additional context for interpreting model behaviour.

5.3 Statistical significance testing **DO NOT TOUCH >CCCCC**

A paired t-test was used to determine whether the differences between the Task 4 configurations were statistically significant. The test was paired because the same ten random seeds were used for each configuration, allowing the results from each seed to be directly matched. The null hypothesis was that there was no significant difference between the paired model results. A significance level of $p < 0.05$ was used. The primary evaluation metrics selected in Section 5.2, recall, PR-AUC, and balanced accuracy, were tested.

	Comparison	Metric	t-statistic	p-value	Statistically significant
0	Default vs Default + undersampling	Recall	-3.223487	1.043093e-02	True
1	Default vs Default + undersampling	PR-AUC	-11.196259	1.386658e-06	True
2	Default vs Default + undersampling	Balanced accuracy	-3.247878	1.003136e-02	True
3	Default + undersampling vs Improved + undersam...	Recall	2.542050	3.160432e-02	True
4	Default + undersampling vs Improved + undersam...	PR-AUC	-20.302492	7.953272e-09	True
5	Default + undersampling vs Improved + undersam...	Balanced accuracy	2.542080	3.160276e-02	True

The paired t-tests showed that introducing undersampling produced statistically significant improvements in recall, PR-AUC, and balanced accuracy compared with the default eLCS. This provides evidence that undersampling had a consistent effect on the model results across the repeated runs rather than the observed differences being explained only by variation between random seeds.

Increasing the eLCS population size and learning iterations also produced statistically significant changes, but these changes were not all improvements. PR-AUC increased significantly, while recall and balanced accuracy significantly decreased compared with the undersampling-only configuration. Therefore, the final improved configuration showed stronger ability to rank positive cases across different thresholds, but weaker positive-case detection at the final classification threshold.

6. Model Comparison

- ☐ Students must compare the proposed improved LCS-based system with the original LCS and at least three conventional machine learning models. The comparison must include:
 - ☐ Original LCS on the raw or minimally processed dataset.
 - ☐ Original LCS on the preprocessed dataset.
 - ☐ Improved LCS-based system on the preprocessed and/or feature-engineered dataset.
 - ☐ At least three conventional non-deep-learning machine learning models.
- ☐ Conventional models may include Random Forest, Support Vector Machine, Decision Tree, Logistic Regression, Naive Bayes, k-Nearest Neighbours, Gaussian Process, or other appropriate non-deep-learning models. Deep-learning models are not permitted for the required model comparison.

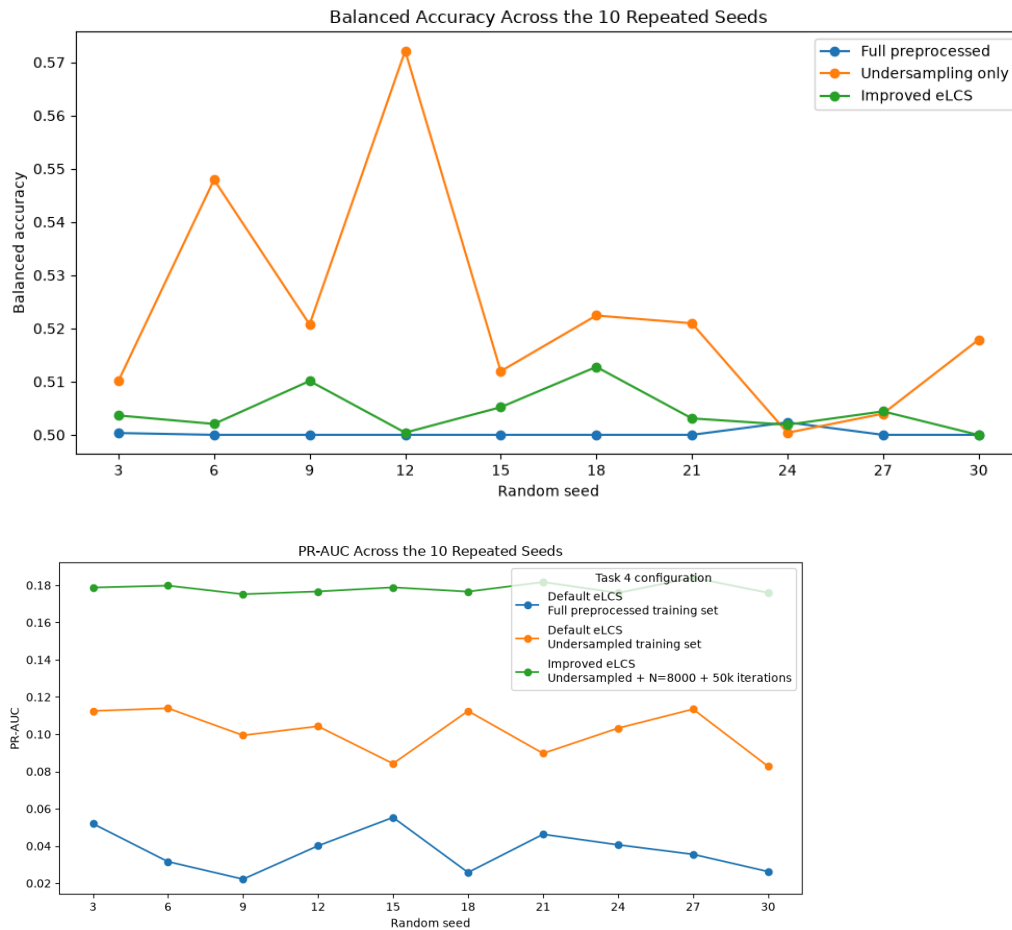
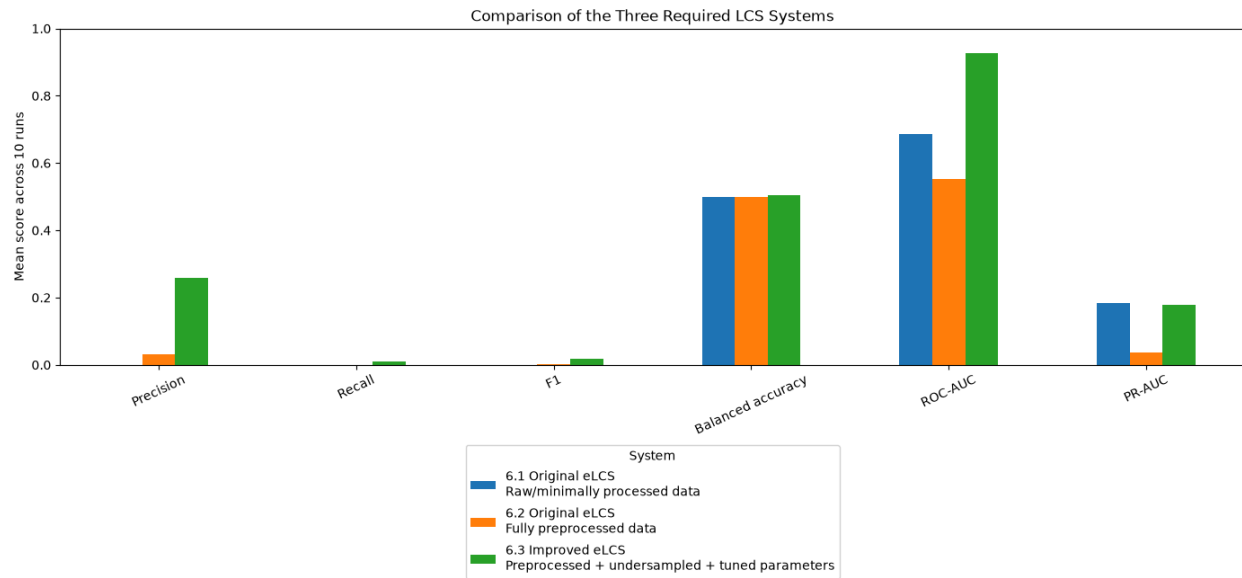


Figure 5.3.2: PR-AUC across the same 10 random seeds for the three eLCS configurations 🤔

yo



6.1 Chosen three conventional models

Logistic regression, decision tree, and random forest would be used for the comparison of the eLCS model prediction results. These three chosen models are all classification models, which allow for a valid comparison because the heart_disease_target is a classification problem. These represent different ways of predicting the target outcome, which would be helpful to compare their result metrics, computational cost, and interpretability.

Logistic Regression

For the logistic regression model, it utilizes weights to form predictions. The weight change is determined by the loss function, which measures how similar the predicted probability is to the actual target value during training. Once the loss value is found using the loss function, the weight would need to be adjusted so that the loss value decreases. Logistic regression is used for comparison since this is a simple classification model which would provide a good benchmark to see if a simpler model is able to achieve a comparable performance to eLCS.

Decision Tree

A decision tree is similar to eLCS, where both utilize if-then rules, but how the rules are formed is different. Decision Trees generate rules by checking which split (condition) gets the purest group (The most dominant class). The group of instances would only stop once it reaches a leaf node (the last node of the tree branch) or a specified limit is introduced. The majority class of that node would be the prediction made. Decision trees have similar Interpretivity towards eLCS; therefore, it has been chosen for the comparison.

Random Forest

Random forest utilizes multiple decision tree predictions as votes, with the prediction with the most common vote being represented as the collective prediction. Each decision tree is trained on a different subset of data; thus, their error could be different, so mistakes in prediction would likely be outvoted. This helps with lowering the instability and overfitting compared to using a single decision tree. Random forest is widely regarded as a strong benchmark model but is hard to interpret. If eLCS can match its predictive performance, it would provide comparable accuracy with greater interpretability.

6.2 Model Setup

```
# Same seeds as Task 4.
SEEDS = [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]

# Settings remain constant across runs.
LOGISTIC_SETTINGS = {
    "solver": "lbfgs",
    "max_iter": 2000,
    "C": 1.0,
    "class_weight": None,
}

TREE_SETTINGS = {
    "criterion": "gini",
    "max_depth": None,
    "min_samples_split": 2,
    "min_samples_leaf": 1,
    "class_weight": None,
}

FOREST_SETTINGS = {
    "n_estimators": 100,
    "criterion": "gini",
    "max_depth": None,
    "min_samples_split": 2,
    "min_samples_leaf": 1,
    "class_weight": None,
    "n_jobs": -1,
}
```

Figure 6.5.1: 3 conventional model specification

The eLCS results of the baseline eLCS, the preprocessed eLCS and the improved eLCS (that contains changes in the hyperparameters and undersampling) are compared with the newly introduced conventional models (logistic regression, decision tree and random forest). These models were tested using the same 80/20 split performed in task 2. These models were also given the same 10 seeds to ensure that the comparison is as fair as possible as well as the same split dataset (the preprocessed dataset with undersampling). The 10 seeds would then be averaged out and find the mean and standard deviation. The parameters of the Logistic regression, decision tree and forest setting were mostly default values, but 2 hyperparameters were changed. The max_iter of the logistic regression were changed because the default was 100, setting it to 2000 ensures as a precaution that the convergence is reached. The other changed

hyperparameter is the `n_jobs` in random forest, `n_jobs` were used to speed up the process of the training so it does not change the outcome of the model.

Even though the baseline and preprocessed eLCS weren't the fairest comparison as they used the dataset with class imbalance, they provide a good reference point of how much the improved eLCS made. The fairest test is against the improved eLCS and the three conventional models trained on the same preprocessed, undersampled data as they have the most similar conditions.

6.3 Model Comparison

	Metric	mean	std	min	max
0	Accuracy	0.952064	0.000000e+00	0.952064	0.952064
1	Precision	0.154800	0.000000e+00	0.154800	0.154800
2	Recall	0.204380	0.000000e+00	0.204380	0.204380
3	F1	0.200479	2.825495e-17	0.200479	0.200479
4	Balanced accuracy	0.625430	0.000000e+00	0.625430	0.625430
5	ROC-AUC	0.876390	1.170279e-16	0.876390	0.876390
6	PR-AUC	0.124865	1.462547e-17	0.124865	0.124865
7	Training seconds	0.246005	5.279268e-02	0.190381	0.350001

Figure 6.51: Summary metric of Logistic Regression

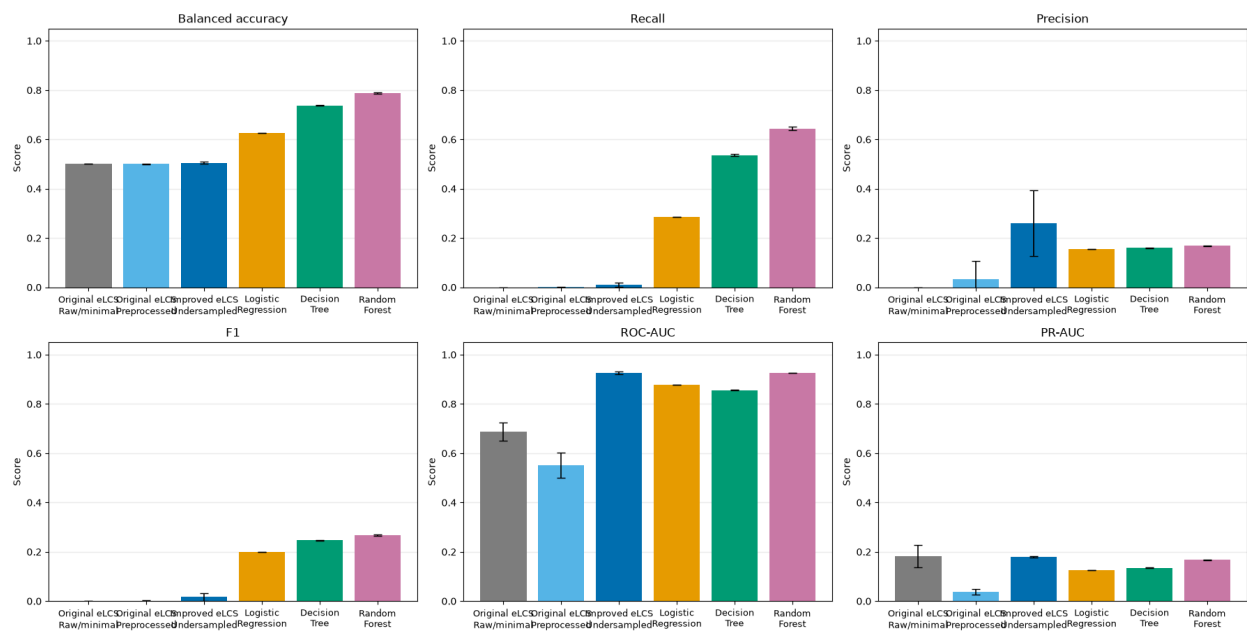
	Metric	mean	std	min	max
0	Accuracy	0.930805	0.000276	0.930192	0.931146
1	Precision	0.160147	0.001253	0.158493	0.162395
2	Recall	0.535823	0.004097	0.532258	0.543294
3	F1	0.246382	0.001804	0.244254	0.250349
4	Balanced accuracy	0.737578	0.002074	0.735623	0.741397
5	ROC-AUC	0.855803	0.001958	0.853382	0.859517
6	PR-AUC	0.134863	0.001837	0.132446	0.137905
7	Training seconds	0.327880	0.162005	0.165402	0.554343

Figure 6.52: Summary metric of Decision Tree

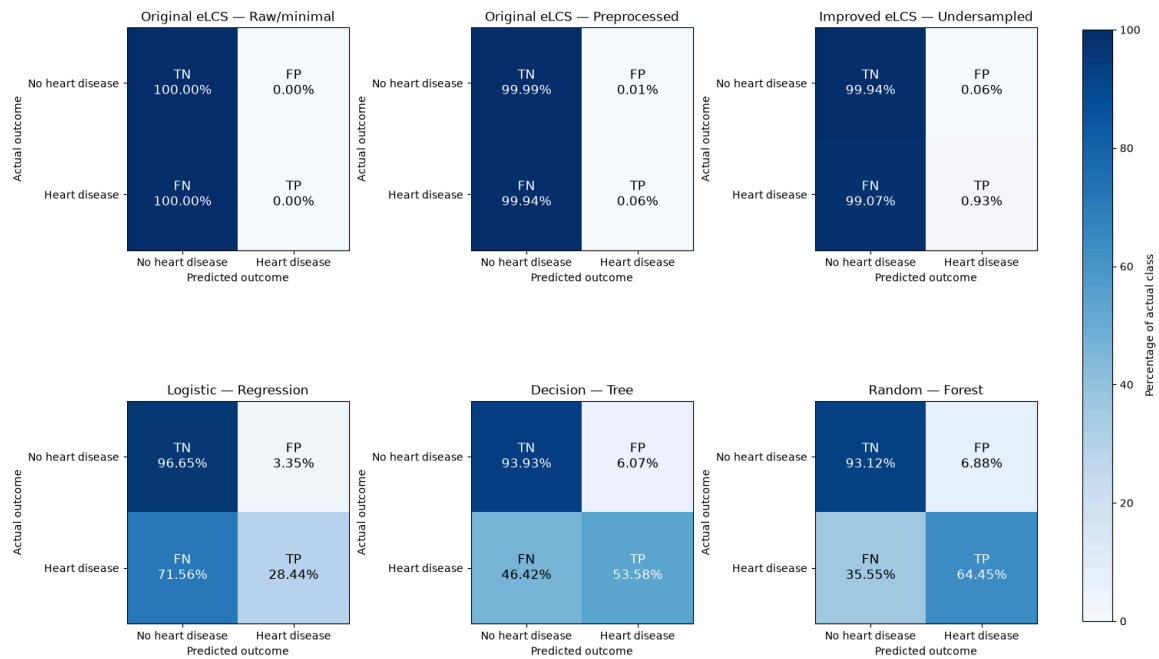
	Metric	mean	std	min	max
0	Accuracy	0.925168	0.000611	0.924584	0.926268
1	Precision	0.168278	0.001461	0.166484	0.169968
2	Recall	0.644482	0.008373	0.629032	0.654499
3	F1	0.266870	0.002320	0.260770	0.268922
4	Balanced accuracy	0.787855	0.003966	0.780024	0.792565
5	ROC-AUC	0.926873	0.000598	0.925958	0.927952
6	PR-AUC	0.167868	0.001214	0.165773	0.168373
7	Training seconds	1.765678	0.271097	1.432392	2.227363

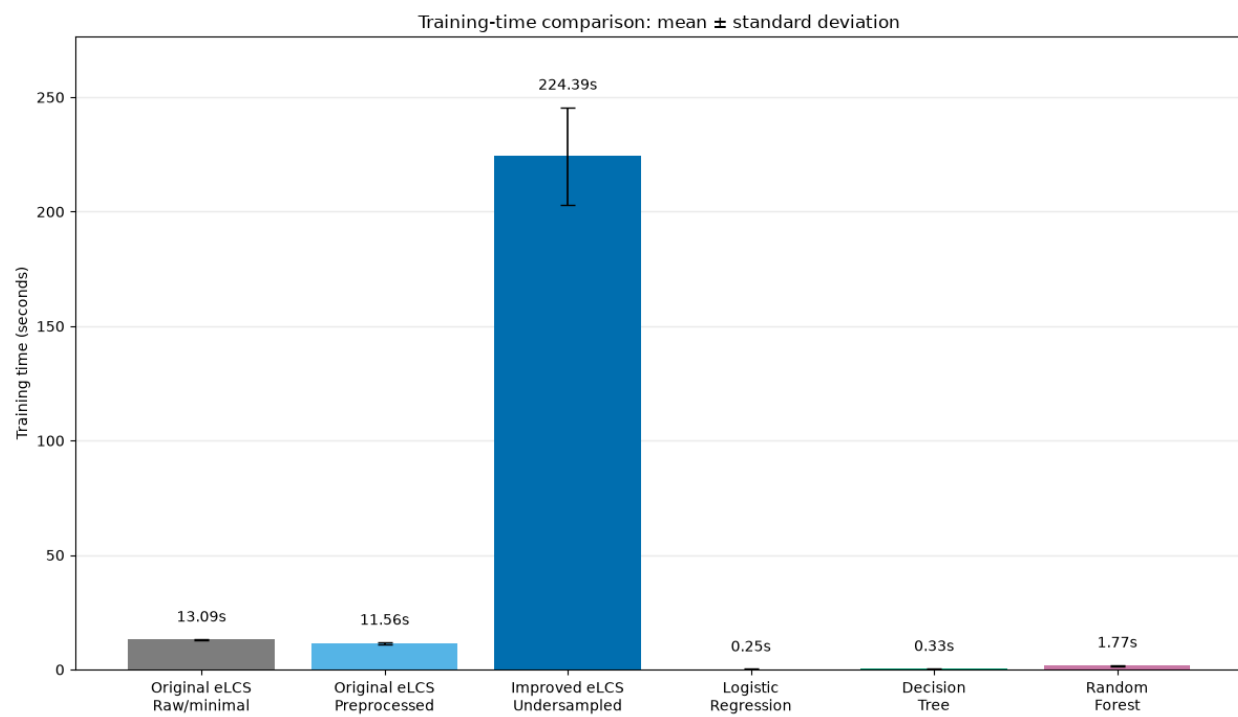
Figure 6.53: Summary metric of Random Forest

Model comparison: mean $\pm$ standard deviation across ten runs



Confusion matrices: mean row-normalised percentages





7. Interpretation of Results and Explainable AI, leo cao :3

- ☐ Students must include a separate section on result interpretation and Explainable AI. Students are required to:
 - ☐ Interpret the main findings from the model comparison.
 - ☐ Explain important LCS rules, feature conditions, or classifier patterns.
 - ☐ Use at least three sample LCS rules/classifiers to discuss the decision-making process.
 - ☐ Discuss how LCS rule-based outputs support interpretability.
 - ☐ Discuss whether the explanations are trustworthy and practically meaningful.

7.1 Interpretation of main findings from the model comparison

7.2 Explanation of key LCS rules / feature conditions / classifier patterns

7.3 At least three sample LCS rules discussed for decision-making insight

7.4 How rule-based outputs support interpretability

7.5 Trustworthiness and practical meaningfulness of the explanations

8. Discussion, Limitations, and Future Work

- ☐ Students must critically discuss the results and the responsible use of the developed system. Students are required to discuss:
 - ☐ Reasons for performance improvement or lack of improvement.
 - ☐ Practical meaning of the results, not only numerical performance.
 - ☐ Limitations due to data quality, dataset size, class imbalance, noise, missingness, or feature relevance.
 - ☐ Future work, such as improved LCS variants, better preprocessing, external validation, or domain expert review.

8.1 Reasons for performance improvement (or lack thereof)

8.2 Practical significance of results beyond raw numbers

8.3 Limitations – data quality, dataset size, class imbalance, noise, missingness, feature relevance

Comparison limitation to note:

"The improved eLCS may have outperformed the original eLCS runs partly because of undersampling, which exposed the model to more minority-class instances, and partly because of its tuned hyperparameters. These two changes were applied together, so their separate effects cannot be identified. Compared with the conventional models, which were trained on the same undersampled data but used default settings, the improved eLCS also had the advantage of tuning, so part of any gain may come from tuning rather than from the eLCS approach itself. Future work could tune the conventional models with a cross-validated search, or run the eLCS with undersampling but default hyperparameters, to separate these effects."

8.4 Future work – improved LCS variants, better preprocessing, external validation, domain expert review

Notes

Final report: 5,000--6,500 words, excluding references and appendices.

Source code or Jupyter Notebook.

GitHub repository link and commit-history evidence.

Contribution statement, including a brief individual reflection from each group member.

Cleaned and feature-engineered dataset, or clear dataset reproduction instructions.

Turnitin similarity report with a similarity score not exceeding 20%.

References

Dataset

- <https://www.kaggle.com/datasets/danishjmeo/281k-patient-dataset-for-multi-disease-prediction?resource=download>

Sub Datasets

- <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset>,
- [kaggle.com/datasets/oktayrdeki/heart-disease](https://www.kaggle.com/datasets/oktayrdeki/heart-disease),
- [kaggle.com/datasets/ankushpanday1/hypertension-risk-prediction-dataset](https://www.kaggle.com/datasets/ankushpanday1/hypertension-risk-prediction-dataset)

Research

Latest heart disease statistics. (n.d.). Heart Foundation NZ.
<https://www.heartfoundation.org.nz/statistics>